参考文献

[1] Lo Y M D, Corbetta N, Chamberlain P F, et al. Presence of fetal DNA in maternal plasma and serum[J]. The Lancet, 1997, 350(9076): 485–487.

[2] Chiu R W K, Chan K C A, Gao Y, et al. Noninvasive prenatal diagnosis of fetal chromosomal aneuploidy by massively parallel genomic sequencing of DNA in maternal plasma[J]. Proceedings of the National Academy of Sciences, 2008, 105(51): 20458–20463.

[3] Wang E, Batey A, Struble C, et al. Gestational age and maternal weight effects on fetal cell-free DNA in maternal plasma[J]. Prenatal Diagnosis, 2013, 33(7): 662–666.

[4] Ashoor G, Syngelaki A, Poon L C Y, et al. Fetal fraction in maternal plasma cell-free DNA at 11–13 weeks' gestation: relation to maternal and fetal characteristics[J]. Ultrasound in Obstetrics & Gynecology, 2013, 41(1): 26–32.

[5] Bianchi D W, Parker R L, Wentworth J, et al. DNA sequencing versus standard prenatal aneuploidy screening[J]. New England Journal of Medicine, 2014, 370(9): 799–808.

[6] Norton M E, Jacobsson B, Swamy G K, et al. Cell-free DNA analysis for noninvasive examination of trisomy[J]. New England Journal of Medicine, 2015, 372(17): 1589–1597.

[7] Klein J P, Moeschberger M L. Survival Analysis: Techniques for Censored and Truncated Data[M]. 2nd ed. New York: Springer, 2003.

[8] 王淑娟, 高志英, 吕艳萍, 等. 孕妇外周血中游离胎儿 DNA 检测在诊断胎儿染色体异常中的应用价值 [J]. 中华妇产科杂志, 2012, 47(11): 808–812.

[9] 赵军红, 代鹏, 朱若男, 等. 2398 例孕妇外周血游离胎儿 DNA 产前筛查阳性结果的验证与分析 [J]. 中华妇产科杂志, 2020, 55(10): 679–684.

[10] 国家卫生计生委办公厅. 孕妇外周血胎儿游离 DNA 产前筛查与诊断技术规范 [S]. 国卫办妇幼发〔2016〕45 号, 2016.

附录

附录 A　补充数据表

本部分汇总正文中受篇幅限制未完整列出的中间结果。表 A.1 为问题一候选混合效应模型的信息准则比较；表 A.2 给出问题二不同分组数下的动态规划决策损失；表 A.3 为问题二达标定义与技术误差设定的单因素敏感性；表 A.4 展示问题三检测失败情景下的分组推荐时点；表 A.5 与表 A.6 分别给出问题四的阳性标签审计与候选特征集的折外性能。

表 A.1　问题一候选混合效应模型的对数似然与信息准则（按 BIC 升序）

候选模型结构	对数似然	AIC	BIC
变点 18 周	-393.73	801.45	835.96
变点 17 周	-396.11	806.23	840.74
变点 16 周	-398.87	811.74	846.24
线性（无变点）	-403.28	818.56	848.14
孕周 × BMI 交互	-401.16	816.33	850.83
变点 15 周	-401.23	816.46	850.97
变点 14 周	-402.61	819.23	853.74
变点 13 周	-403.16	820.32	854.82
加入年龄、身高	-400.19	816.38	855.82

表 A.2　问题二分组数 K 的动态规划总损失与 BMI 切点（普通判定口径）

可靠度 ρ	分组数 K	总损失	损失下降占比	BMI 切点
0.80	1	122.19	0.0%	—
0.80	2	74.67	49.7%	31.77
0.80	3	46.95	78.7%	31.03, 35.03
0.80	4	36.66	89.5%	30.91, 33.62, 36.10
0.80	5	30.97	95.4%	29.53, 31.03, 33.62, 36.10
0.80	6	26.62	100.0%	29.53, 30.86, 32.70, 34.26, 36.10
0.90	1	112.03	0.0%	—
0.90	2	59.85	58.5%	33.81
0.90	3	38.88	82.0%	31.03, 34.82
0.90	4	29.27	92.7%	30.76, 33.62, 36.10
0.90	5	25.73	96.7%	29.93, 31.43, 33.62, 36.10
0.90	6	22.77	100.0%	29.27, 30.64, 32.19, 33.62, 36.10

表 A.3　问题二达标阈值、可信水平与误差倍数的单因素敏感性（$\rho = 0.80$）

判定方式	达标阈值	可信水平 η	误差倍数	最优分布	BMI 系数	单位 BMI 时间比
硬判定	0.035	0.025	1.00	对数 logistic	0.0445	1.0455
可信判定	0.035	0.025	1.00	对数 logistic	0.1403	1.1506
硬判定	0.040	0.025	1.00	对数正态	0.0392	1.0400
可信判定	0.040	0.025	1.00	对数 logistic	0.0830	1.0865
硬判定	0.045	0.025	1.00	对数正态	0.0365	1.0372
可信判定	0.045	0.025	1.00	Weibull	0.0520	1.0533
可信判定	0.040	0.010	1.00	对数 logistic	0.1058	1.1116
可信判定	0.040	0.050	1.00	对数 logistic	0.0746	1.0775
可信判定	0.040	0.100	1.00	Weibull	0.0484	1.0496
可信判定	0.040	0.025	0.75	Weibull	0.0560	1.0576
可信判定	0.040	0.025	1.25	对数 logistic	0.1041	1.1098
可信判定	0.040	0.025	1.50	对数 logistic	0.1047	1.1104

表 A.4　问题三检测失败概率与复测延迟情景下的分组推荐时点

可靠度 ρ	失败概率	复测延迟（周）	第 1 组	第 2 组	第 3 组	第 4 组
0.80	0%	1	11 周 +4 天	12 周 +6 天	14 周 +0 天	16 周 +2 天
0.80	0%	2	11 周 +4 天	12 周 +6 天	14 周 +0 天	16 周 +2 天
0.80	5%	1	11 周 +4 天	12 周 +6 天	14 周 +0 天	16 周 +3 天
0.80	5%	2	11 周 +5 天	13 周 +0 天	14 周 +1 天	16 周 +3 天
0.80	10%	1	11 周 +5 天	13 周 +0 天	14 周 +1 天	16 周 +3 天
0.80	10%	2	11 周 +6 天	13 周 +0 天	14 周 +2 天	16 周 +4 天
0.80	15%	1	11 周 +5 天	13 周 +0 天	14 周 +1 天	16 周 +3 天
0.80	15%	2	11 周 +6 天	13 周 +1 天	14 周 +2 天	16 周 +5 天
0.90	0%	1	14 周 +5 天	16 周 +2 天	17 周 +6 天	22 周 +1 天
0.90	0%	2	14 周 +5 天	16 周 +2 天	17 周 +6 天	22 周 +1 天
0.90	5%	1	14 周 +5 天	16 周 +3 天	18 周 +0 天	22 周 +2 天
0.90	5%	2	14 周 +6 天	16 周 +3 天	18 周 +0 天	22 周 +2 天
0.90	10%	1	14 周 +6 天	16 周 +3 天	18 周 +0 天	22 周 +2 天
0.90	10%	2	15 周 +0 天	16 周 +4 天	18 周 +1 天	22 周 +3 天
0.90	15%	1	14 周 +6 天	16 周 +4 天	18 周 +0 天	22 周 +2 天
0.90	15%	2	15 周 +0 天	16 周 +5 天	18 周 +2 天	22 周 +4 天

表 A.5　问题四女胎阳性标签规模与 $|Z| \geq 3$ 规则覆盖审计

异常类型	阳性记录数	阳性孕妇数	$\lvert Z\rvert \geq 3$ 命中比例	阳性 Z 值范围
T13	23	18	4.3%	[-2.81, 3.81]
T18	46	30	0.0%	[-0.90, 2.97]
T21	13	12	0.0%	[-2.39, 1.33]

表 A.6　问题四候选特征集折外性能（多种子加权均值，弹性网络逻辑回归）

任务	特征集	特征数	ROC-AUC	PR-AUC	F1	F1 的 95% 区间
ANY	仅 Z 值	9	0.485	0.123	0.193	[0.169, 0.207]
ANY	核心测序特征	29	0.784	0.435	0.426	[0.377, 0.459]
ANY	核心 + 手工特征	64	0.743	0.387	0.412	[0.337, 0.477]
ANY	加入纵向特征	157	0.703	0.308	0.325	[0.307, 0.355]
ANY	去除孕妇特征	149	0.686	0.320	0.327	[0.275, 0.390]
ANY	去除质控特征	87	0.675	0.297	0.292	[0.252, 0.345]
T13	仅 Z 值	9	0.367	0.034	0.065	[0.054, 0.075]
T13	核心测序特征	29	0.823	0.230	0.312	[0.237, 0.374]
T13	核心 + 手工特征	64	0.807	0.184	0.234	[0.134, 0.272]
T13	加入纵向特征	157	0.828	0.210	0.270	[0.187, 0.349]
T13	去除孕妇特征	149	0.846	0.216	0.272	[0.237, 0.343]
T13	去除质控特征	87	0.643	0.075	0.102	[0.043, 0.162]
T18	仅 Z 值	9	0.571	0.094	0.161	[0.130, 0.185]
T18	核心测序特征	29	0.850	0.517	0.461	[0.383, 0.537]
T18	核心 + 手工特征	64	0.830	0.475	0.471	[0.422, 0.514]
T18	加入纵向特征	157	0.789	0.379	0.386	[0.299, 0.469]
T18	去除孕妇特征	149	0.789	0.401	0.426	[0.355, 0.516]
T18	去除质控特征	87	0.723	0.285	0.341	[0.268, 0.408]
T21	仅 Z 值	9	0.334	0.018	0.042	[0.033, 0.047]
T21	核心测序特征	29	0.497	0.179	0.059	[0.044, 0.091]
T21	核心 + 手工特征	64	0.453	0.157	0.042	[0.018, 0.054]
T21	加入纵向特征	157	0.416	0.041	0.025	[0.000, 0.050]
T21	去除孕妇特征	149	0.387	0.035	0.020	[0.000, 0.045]
T21	去除质控特征	87	0.376	0.020	0.040	[0.029, 0.047]

附录 B　核心源代码

本部分给出模型的核心求解程序。代码已脱敏处理，数据文件以相对路径“附件.xlsx”读入，不含任何本地绝对路径；关键步骤处添加了必要注释。运行环境为 Python 3.11 及以上版本，依赖 numpy、pandas、scipy、statsmodels 等开源库。固定随机种子 `SEED = 20250904`，全部结果可复现。

`nipt\_core.py`（核心函数库：数据准备、技术误差估计、区间删失 AFT、动态规划分组）

```
# -*- coding: utf-8 -*-
#
================================================================================
# nipt_core.py  NIPT 建模核心函数库
# 功能：数据读取与清洗、技术误差合并估计、达标状态判定、区间删失构造、
#       区间删失 AFT 模型拟合（对数正态 / 对数 logistic / Weibull）、
#       BMI 分组的一维精确动态规划与排程评估。
# 说明：输入数据为竞赛附件（相对路径），代码不含任何本地绝对路径。
#
================================================================================

from __future__ import annotations

import hashlib
import json
import math
import re
import warnings
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Sequence

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.special import expit, log_ndtr
from scipy.stats import norm


SEED = 20250904
Y_THRESHOLD = 0.04
EPS = 1e-12

MALE_SHEET = "男胎检测数据"
FEMALE_SHEET = "女胎检测数据"
```

```

QUALITY_COLUMNS = [
    " 原始读段数",
    " 在参考基因组上比对的比例",
    " 重复读段的比例",
    " 唯一比对的读段数",
    "GC 含量",
    "13 号染色体的 GC 含量",
    "18 号染色体的 GC 含量",
    "21 号染色体的 GC 含量",
    " 被过滤掉读段数的比例",
]



# ---------- 基础解析工具：孕周、日期、孕次的统一解析 ----------
def parse_ga(value: object) -> float:
    text = str(value).strip().lower()
    match = re.fullmatch(r"(\d+)w(?:\+(\d+))?", text)
    if not match:
        raise ValueError(f"Unrecognized gestational age: {value!r}")
    weeks = int(match.group(1))
    days = int(match.group(2) or 0)
    if days < 0 or days > 6:
        raise ValueError(f"Invalid gestational days: {value!r}")
    return weeks + days / 7.0


def parse_date(value: object) -> pd.Timestamp:
    if isinstance(value, (int, np.integer)):
        return pd.to_datetime(str(int(value)), format="%Y%m%d")
    if isinstance(value, float) and value.is_integer():
        return pd.to_datetime(str(int(value)), format="%Y%m%d")
    return pd.to_datetime(value)


def gravidity_numeric(value: object) -> float:
    text = str(value).strip()
    if text in {"≥3", ">=3", "3 及以上"}:
        return 3.0
    return float(text)


def to_jsonable(value: Any) -> Any:
```

```
    if isinstance(value, dict):
        return {str(k): to_jsonable(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [to_jsonable(v) for v in value]
    if isinstance(value, (np.integer,)):
        return int(value)
    if isinstance(value, (np.floating,)):
        return None if not np.isfinite(value) else float(value)
    if isinstance(value, np.ndarray):
        return to_jsonable(value.tolist())
    if isinstance(value, pd.Timestamp):
        return value.isoformat()
    if isinstance(value, Path):
        return str(value)
    if pd.isna(value) if np.isscalar(value) else False:
        return None
    return value


def write_json(path: Path, payload: Any) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(
        json.dumps(to_jsonable(payload), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def week_day(value: float) -> str:
    if not np.isfinite(value):
        return "NA"
    weeks = int(np.floor(value))
    days = int(np.round(7 * (value - weeks)))
    if days == 7:
        weeks += 1
        days = 0
    return f"{weeks} 周 +{days} 天"
```

```python


@dataclass
class PreparedData:
    male_raw: pd.DataFrame
    female_raw: pd.DataFrame
    male_visits: pd.DataFrame
    male_baseline: pd.DataFrame
    sigma_tech: float
    sigma_tech_strict: float
    technical_summary: dict[str, Any]



# ---------- 技术误差估计：同一采血事件复测浓度的合并标准差 ----------
def pooled_technical_sd(
    frame: pd.DataFrame, key: Sequence[str], value_col: str
) -> tuple[float, int, int, list[float]]:
    pooled_ss = 0.0
    pooled_df = 0
    repeated = 0
    pair_diffs: list[float] = []
    for _, group in frame.groupby(list(key), dropna=False, sort=False):
        if len(group) <= 1:
            continue
        repeated += 1
        values = group[value_col].to_numpy(float)
        pooled_ss += float(np.square(values - values.mean()).sum())
        pooled_df += len(values) - 1
        for i in range(len(values)):
            for j in range(i + 1, len(values)):
                pair_diffs.append(abs(float(values[i] - values[j])))
    sigma = float(np.sqrt(pooled_ss / pooled_df))
    return sigma, pooled_df, repeated, pair_diffs



# ---------- 数据准备：读入附件、构造采血事件层与孕妇基线层 ----------
def prepare_data(data_path: Path) -> PreparedData:
    male = pd.read_excel(data_path, sheet_name=MALE_SHEET)
    female = pd.read_excel(data_path, sheet_name=FEMALE_SHEET)
    male.columns = male.columns.str.strip()
    female.columns = female.columns.str.strip()

```

```
    for frame in (male, female):
        frame["ga"] = frame[" 检测孕周"].map(parse_ga)
        frame["date_norm"] = frame[" 检测日期"].map(parse_date)
        frame["gravidity_num"] = frame[" 怀孕次数"].map(gravidity_numeric)
        frame["ivf_iui"] = frame["IVF 妊娠"].ne(" 自然受孕").astype(int)
        frame["ivf"] = frame["IVF 妊娠"].str.contains("IVF",
        regex=False).astype(int)
        frame["iui"] = frame["IVF 妊娠"].str.contains("IUI",
        regex=False).astype(int)

    visit_key = [" 孕妇代码", " 检测抽血次数", "ga"]
    strict_key = [" 孕妇代码", "date_norm", " 检测抽血次数", "ga"]
    sigma, tech_df, repeated, pair_diffs = pooled_technical_sd(
        male, visit_key, "Y 染色体浓度"
    )
    sigma_strict, tech_df_strict, repeated_strict, _ =
    pooled_technical_sd(
        male, strict_key, "Y 染色体浓度"
    )

    ordered = male.sort_values([" 孕妇代码", "ga", "date_norm", " 序号"])
    agg: dict[str, tuple[str, str]] = {
        "date_norm": ("date_norm", "min"),
        "y": ("Y 染色体浓度", "median"),
        "bmi": (" 孕妇 BMI", "median"),
        "weight": (" 体重", "median"),
        "age": (" 年龄", "first"),
        "height": (" 身高", "first"),
        "gravidity": ("gravidity_num", "first"),
        "parity": (" 生产次数", "first"),
        "ivf_iui": ("ivf_iui", "first"),
        "ivf": ("ivf", "first"),
        "iui": ("iui", "first"),
        "n_tech": ("Y 染色体浓度", "size"),
        "y_mean": ("Y 染色体浓度", "mean"),
        "y_min": ("Y 染色体浓度", "min"),
        "y_max": ("Y 染色体浓度", "max"),
    }
    for source in QUALITY_COLUMNS:
        safe = {
            " 原始读段数": "raw_reads",
            " 在参考基因组上比对的比例": "map_rate",
            " 重复读段的比例": "duplicate_rate",
            " 唯一比对的读段数": "unique_reads",
```

```
            "GC 含量": "gc",
            "13 号染色体的 GC 含量": "gc13",
            "18 号染色体的 GC 含量": "gc18",
            "21 号染色体的 GC 含量": "gc21",
            " 被过滤掉读段数的比例": "filter_rate",
        }[source]
        agg[safe] = (source, "median")
    visits = (
        ordered.groupby(visit_key, as_index=False, dropna=False)
        .agg(**agg)
        .sort_values([" 孕妇代码", "ga", "date_norm"])
        .reset_index(drop=True)
    )
    visits["y_range"] = visits["y_max"] - visits["y_min"]

    first_rows = (
        visits.sort_values([" 孕妇代码", "ga", "date_norm"])
        .groupby(" 孕妇代码", sort=False)
        .first()
        .reset_index()
    )
    baseline_cols = [
        " 孕妇代码",
        "ga",
        "date_norm",
        "bmi",
        "weight",
        "age",
        "height",
        "gravidity",
        "parity",
        "ivf_iui",
        "ivf",
        "iui",
    ]
    baseline = first_rows[baseline_cols].rename(
        columns={
            "date_norm": "first_date",
            "ga": "first_ga",
            "bmi": "first_bmi",
            "weight": "first_weight",
        }
    )
    visits = visits.merge(
```

```
        baseline[[" 孕妇代码", "first_ga", "first_bmi", "first_weight"]],
        on=" 孕妇代码",
        how="left",
        validate="many_to_one",
    )
    visits["delta_bmi"] = visits["bmi"] - visits["first_bmi"]

    technical_summary = {
        "male_rows": len(male),
        "male_women": int(male[" 孕妇代码"].nunique()),
        "male_visits": len(visits),
        "female_rows": len(female),
        "female_women": int(female[" 孕妇代码"].nunique()),
        "repeat_groups": repeated,
        "repeat_df": tech_df,
        "strict_repeat_groups": repeated_strict,
        "strict_repeat_df": tech_df_strict,
        "sigma_tech": sigma,
        "sigma_tech_strict": sigma_strict,
        "pairwise_abs_diff_median": float(np.median(pair_diffs)),
        "y_range": [float(male["Y 染色体浓度"].min()), float(male["Y 染色
体浓度"].max())],
        "ga_range": [float(visits["ga"].min()),
        float(visits["ga"].max())],
    }
    return PreparedData(
        male_raw=male,
        female_raw=female,
        male_visits=visits,
        male_baseline=baseline,
        sigma_tech=sigma,
        sigma_tech_strict=sigma_strict,
        technical_summary=technical_summary,
    )


MEDIAN_SE_FACTORS = {1: 1.0, 2: 2 ** -0.5, 3: 0.67017}


def median_se_factor(replicates: int) -> float:
    if replicates in MEDIAN_SE_FACTORS:
        return MEDIAN_SE_FACTORS[replicates]
    return math.sqrt(math.pi / (2 * replicates))

```

```python


# ---------- 达标状态判定：硬判定 / 可信区间判定 / 扰动判定 ----------
def classify_visit_states(
    visits: pd.DataFrame,
    sigma_tech: float,
    *,
    threshold: float = Y_THRESHOLD,
    mode: str = "hard",
    eta: float = 0.025,
    error_multiplier: float = 1.0,
    rng: np.random.Generator | None = None,
) -> pd.DataFrame:
    result = visits.copy()
    factors = result["n_tech"].map(lambda x:
    median_se_factor(int(x))).to_numpy(float)
    result["se_tech"] = sigma_tech * factors * error_multiplier
    y = result["y"].to_numpy(float)
    if mode == "hard":
        states = (y >= threshold).astype(int)
    elif mode == "credible":
        z = norm.ppf(1 - eta)
        lower = y - z * result["se_tech"].to_numpy(float)
        upper = y + z * result["se_tech"].to_numpy(float)
        states = np.where(lower >= threshold, 1, np.where(upper <
        threshold, 0, -1))
    elif mode == "perturbed":
        if rng is None:
            raise ValueError("rng is required for perturbed states")
        simulated = y + rng.normal(0, result["se_tech"].to_numpy(float))
        result["y_perturbed"] = simulated
        states = (simulated >= threshold).astype(int)
    else:
        raise ValueError(f"Unknown state mode: {mode}")
    result["state"] = states
    return result



# ---------- 由达标状态序列构造首次达标时间的删失区间 ----------
def construct_intervals(
    state_visits: pd.DataFrame,
    baseline: pd.DataFrame,
    *,
```

```
    exclude_conflicts: bool = False,
) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    baseline_index = baseline.set_index(" 孕妇代码")
    for code, group in state_visits.groupby(" 孕妇代码", sort=False):
        group = group.sort_values(["ga",
        "date_norm"]).reset_index(drop=True)
        states = group["state"].to_numpy(int)
        positive = np.flatnonzero(states == 1)
        negative = np.flatnonzero(states == 0)
        conflict = False
        if len(positive):
            first_positive = int(positive[0])
            conflict = bool(np.any(states[first_positive + 1 :] == 0))
            before_negative = negative[negative < first_positive]
            if len(before_negative):
                left = float(group.iloc[int(before_negative[-1])]["ga"])
                right = float(group.iloc[first_positive]["ga"])
                censor_type = "interval"
            else:
                left = 0.0
                right = float(group.iloc[first_positive]["ga"])
                censor_type = "left"
        elif len(negative):
            left = float(group.iloc[int(negative[-1])]["ga"])
            right = np.inf
            censor_type = "right"
        else:
            left = np.nan
            right = np.nan
            censor_type = "uninformative"
        if exclude_conflicts and conflict:
            continue
        base = baseline_index.loc[code]
        rows.append(
            {
                "code": code,
                "left": left,
                "right": right,
                "type": censor_type,
                "state_conflict": int(conflict),
                "n_visits": len(group),
                "n_uncertain": int(np.sum(states == -1)),
                "first_bmi": float(base["first_bmi"]),
```

```
                "first_weight": float(base["first_weight"]),
                "age": float(base["age"]),
                "height": float(base["height"]),
                "gravidity": float(base["gravidity"]),
                "parity": float(base["parity"]),
                "ivf_iui": int(base["ivf_iui"]),
                "ivf": int(base["ivf"]),
                "iui": int(base["iui"]),
                "first_ga": float(base["first_ga"]),
            }
        )
    return pd.DataFrame(rows)


def logdiffexp(log_large: np.ndarray, log_small: np.ndarray) ->
np.ndarray:
    delta = np.minimum(log_small - log_large, -np.finfo(float).eps)
    return log_large + np.log1p(-np.exp(delta))



# ---------- AFT 误差分布的对数 CDF / 生存函数 / 分位数 ----------
def aft_logcdf(z: np.ndarray, distribution: str) -> np.ndarray:
    if distribution == "lognormal":
        return log_ndtr(z)
    if distribution == "loglogistic":
        return -np.logaddexp(0.0, -z)
    if distribution == "weibull":
        ez = np.exp(np.clip(z, -700, 50))
        return np.log(np.clip(-np.expm1(-ez), 1e-300, 1.0))
    raise ValueError(distribution)


def aft_logsurvival(z: np.ndarray, distribution: str) -> np.ndarray:
    if distribution == "lognormal":
        return log_ndtr(-z)
    if distribution == "loglogistic":
        return -np.logaddexp(0.0, z)
    if distribution == "weibull":
        return -np.exp(np.clip(z, -700, 50))
    raise ValueError(distribution)


def aft_error_quantile(probability: float, distribution: str) -> float:
```

```
    if distribution == "lognormal":
        return float(norm.ppf(probability))
    if distribution == "loglogistic":
        return float(np.log(probability / (1 - probability)))
    if distribution == "weibull":
        return float(np.log(-np.log(1 - probability)))
    raise ValueError(distribution)


@dataclass
class AFTFit:
    distribution: str
    feature_cols: list[str]
    coefficients: np.ndarray
    sigma: float
    means: np.ndarray
    scales: np.ndarray
    neg_loglik: float
    success: bool
    message: str
    n: int
    ridge: float = 0.0

    @property
    def parameter_count(self) -> int:
        return len(self.coefficients) + 1

    @property
    def aic(self) -> float:
        return 2 * self.neg_loglik + 2 * self.parameter_count

    @property
    def bic(self) -> float:
        return 2 * self.neg_loglik + self.parameter_count *
        np.log(self.n)

    def standardized_features(self, frame: pd.DataFrame) -> np.ndarray:
        if not self.feature_cols:
            return np.empty((len(frame), 0))
        raw = frame[self.feature_cols].to_numpy(float)
        return (raw - self.means) / self.scales

    def linear_predictor(self, frame: pd.DataFrame) -> np.ndarray:
        x = self.standardized_features(frame)
```

```
        design = np.column_stack([np.ones(len(frame)), x])
        return design @ self.coefficients

    def cdf(self, time: float | np.ndarray, frame: pd.DataFrame) ->
    np.ndarray:
        time_array = np.asarray(time, dtype=float)
        eta = self.linear_predictor(frame)
        z = (np.log(time_array) - eta) / self.sigma
        return np.exp(aft_logcdf(z, self.distribution))

    def quantile(self, probability: float, frame: pd.DataFrame) ->
    np.ndarray:
        q = aft_error_quantile(probability, self.distribution)
        return np.exp(self.linear_predictor(frame) + self.sigma * q)

    def to_dict(self) -> dict[str, Any]:
        return {
            "distribution": self.distribution,
            "feature_cols": self.feature_cols,
            "coefficients": self.coefficients,
            "sigma": self.sigma,
            "means": self.means,
            "scales": self.scales,
            "neg_loglik": self.neg_loglik,
            "aic": self.aic,
            "bic": self.bic,
            "success": self.success,
            "message": self.message,
            "n": self.n,
            "ridge": self.ridge,
        }


# ---------- 区间删失似然：左、右、区间删失三类贡献 ----------
def interval_loglik_contributions(
    params: np.ndarray,
    x: np.ndarray,
    left: np.ndarray,
    right: np.ndarray,
    distribution: str,
) -> np.ndarray:
    beta = params[:-1]
    sigma = float(np.exp(params[-1]))
```

```
    eta = np.column_stack([np.ones(len(x)), x]) @ beta
    left_censored = left == 0
    right_censored = np.isinf(right)
    interval_censored = ~(left_censored | right_censored)
    result = np.empty(len(left), dtype=float)
    if np.any(left_censored):
        z_right = (np.log(right[left_censored]) - eta[left_censored]) /
        sigma
        result[left_censored] = aft_logcdf(z_right, distribution)
    if np.any(right_censored):
        z_left = (np.log(left[right_censored]) - eta[right_censored]) /
        sigma
        result[right_censored] = aft_logsurvival(z_left, distribution)
    if np.any(interval_censored):
        z_left = (np.log(left[interval_censored]) -
        eta[interval_censored]) / sigma
        z_right = (np.log(right[interval_censored]) -
        eta[interval_censored]) / sigma
        log_left = aft_logcdf(z_left, distribution)
        log_right = aft_logcdf(z_right, distribution)
        result[interval_censored] = logdiffexp(log_right, log_left)
    return result



# ---------- AFT 模型拟合：L-BFGS-B 多起点优化，支持岭惩罚 ----------
def fit_aft(
    frame: pd.DataFrame,
    feature_cols: Sequence[str],
    *,
    distribution: str = "lognormal",
    nonnegative_features: Iterable[str] = (),
    ridge: float = 0.0,
    starts: Sequence[np.ndarray] | None = None,
) -> AFTFit:
    usable = frame.loc[
        frame["type"].isin(["left", "interval", "right"])
    ].copy()
    feature_cols = list(feature_cols)
    if feature_cols:
        raw = usable[feature_cols].to_numpy(float)
        means = np.nanmean(raw, axis=0)
        scales = np.nanstd(raw, axis=0, ddof=0)
        scales = np.where(scales < 1e-8, 1.0, scales)
```

```
        raw = np.where(np.isfinite(raw), raw, means)
        x = (raw - means) / scales
    else:
        means = np.empty(0)
        scales = np.empty(0)
        x = np.empty((len(usable), 0))
    left = usable["left"].to_numpy(float)
    right = usable["right"].to_numpy(float)
    p = 1 + len(feature_cols)

    def objective(params: np.ndarray) -> float:
        values = interval_loglik_contributions(
            params, x, left, right, distribution
        )
        if not np.all(np.isfinite(values)):
            return 1e100
        penalty = ridge * float(np.square(params[1:p]).sum()) / 2
        return -float(values.sum()) + penalty

    bounds: list[tuple[float, float]] = [(-2.0, 6.0)]
    nonnegative = set(nonnegative_features)
    for name in feature_cols:
        bounds.append((0.0, 4.0) if name in nonnegative else (-4.0, 4.0))
    bounds.append((-4.0, 2.0))
    if starts is None:
        base = np.zeros(p + 1)
        base[0] = 2.2
        base[-1] = np.log(0.5)
        starts = [
            base,
            base + np.r_[0.2, np.zeros(p - 1), -0.3],
            base + np.r_[-0.2, np.zeros(p - 1), 0.3],
        ]
    fits = [
        minimize(
            objective,
            np.asarray(start, dtype=float),
            method="L-BFGS-B",
            bounds=bounds,
            options={"maxiter": 1500, "ftol": 1e-11},
        )
        for start in starts
    ]
    fit = min(fits, key=lambda item: item.fun)
```

```
    coefficients = fit.x[:-1].copy()
    sigma = float(np.exp(fit.x[-1]))
    return AFTFit(
        distribution=distribution,
        feature_cols=feature_cols,
        coefficients=coefficients,
        sigma=sigma,
        means=means,
        scales=scales,
        neg_loglik=float(fit.fun),
        success=bool(fit.success),
        message=str(fit.message),
        n=len(usable),
        ridge=ridge,
    )


def evaluate_aft_loglik(fit: AFTFit, frame: pd.DataFrame) -> np.ndarray:
    usable = frame.loc[
        frame["type"].isin(["left", "interval", "right"])
    ].copy()
    x = fit.standardized_features(usable)
    params = np.r_[fit.coefficients, np.log(fit.sigma)]
    return interval_loglik_contributions(
        params,
        x,
        usable["left"].to_numpy(float),
        usable["right"].to_numpy(float),
        fit.distribution,
    )


def shuffled_group_folds(
    groups: Sequence[Any],
    *,
    n_splits: int,
    seed: int,
) -> list[tuple[np.ndarray, np.ndarray]]:
    groups_array = np.asarray(groups)
    unique = np.unique(groups_array)
    rng = np.random.default_rng(seed)
    shuffled = unique.copy()
    rng.shuffle(shuffled)
    bins = np.array_split(shuffled, n_splits)
```

```
    result = []
    for test_groups in bins:
        test_mask = np.isin(groups_array, test_groups)
        result.append((np.flatnonzero(~test_mask),
        np.flatnonzero(test_mask)))
    return result


def weighted_mean(values: np.ndarray, weights: np.ndarray) -> float:
    values = np.asarray(values, dtype=float)
    weights = np.asarray(weights, dtype=float)
    return float(np.sum(values * weights) / np.sum(weights))


def per_group_row_weights(groups: Sequence[Any]) -> np.ndarray:
    series = pd.Series(np.asarray(groups))
    counts = series.value_counts()
    return series.map(lambda x: 1.0 / counts[x]).to_numpy(float).copy()


def exact_upper_quantile(values: np.ndarray, probability: float) ->
float:
    finite = np.asarray(values, dtype=float)
    order = np.sort(finite)
    rank = int(np.ceil(probability * len(order))) - 1
    rank = min(max(rank, 0), len(order) - 1)
    return float(order[rank])


@dataclass
class DPSegment:
    start: int
    end: int
    n: int
    bmi_min: float
    bmi_max: float
    raw_time: float
    operational_time: float
    coverage: float
    cost: float


@dataclass
class DPPolicy:
```

```
    groups: int
    alpha: float
    min_size: int
    total_cost: float
    segments: list[DPSegment]
    cutpoints: list[float]
    sorted_order: np.ndarray

    def to_dict(self) -> dict[str, Any]:
        return {
            "groups": self.groups,
            "alpha": self.alpha,
            "min_size": self.min_size,
            "total_cost": self.total_cost,
            "cutpoints": self.cutpoints,
            "segments": [segment.__dict__ for segment in self.segments],
        }



# ---------- 一维精确动态规划：分位数损失下的最优 BMI 分组 ----------
def exact_dp_policy(
    bmi: np.ndarray,
    tau: np.ndarray,
    *,
    groups: int,
    alpha: float,
    min_size: int,
    lower_week: float = 10.0,
    upper_week: float = 25.0,
) -> DPPolicy:
    bmi = np.asarray(bmi, dtype=float)
    tau = np.asarray(tau, dtype=float)
    # 按 BMI 升序排列，分组只在该序上切分，保证组间单调
    order = np.argsort(bmi, kind="stable")
    b = bmi[order]
    t = tau[order]
    n = len(b)
    cost = np.full((n, n), np.inf)
    group_time = np.full((n, n), np.nan)
    raw_time = np.full((n, n), np.nan)
    allowed_start = np.ones(n, dtype=bool)
    allowed_start[1:] = b[1:] > b[:-1]
    for start in range(n):
```

```
        if start > 0 and not allowed_start[start]:
            continue
        for end in range(start + min_size - 1, n):
            if end < n - 1 and b[end] == b[end + 1]:
                continue
            values = t[start : end + 1]
            q = exact_upper_quantile(values, alpha)
            operational = max(lower_week, q)
            if operational > upper_week or not np.isfinite(operational):
                continue
            residual = values - operational
            check = residual * (alpha - (residual < 0).astype(float))
            cost[start, end] = float(np.sum(check))
            raw_time[start, end] = q
            group_time[start, end] = operational
    # dp[g, e]：前 e+1 名孕妇切成 g+1 组的最小累计分位数损失
    dp = np.full((groups, n), np.inf)
    previous = np.full((groups, n), -1, dtype=int)
    dp[0, :] = cost[0, :]
    for g in range(1, groups):
        min_end = (g + 1) * min_size - 1
        for end in range(min_end, n):
            for prev_end in range(g * min_size - 1, end):
                start = prev_end + 1
                candidate = dp[g - 1, prev_end] + cost[start, end]
                if candidate < dp[g, end]:
                    dp[g, end] = candidate
                    previous[g, end] = prev_end
    if not np.isfinite(dp[groups - 1, n - 1]):
        raise ValueError(f"No feasible DP policy for K={groups},
        alpha={alpha}")
    segments: list[DPSegment] = []
    end = n - 1
    for g in range(groups - 1, -1, -1):
        prev_end = int(previous[g, end]) if g > 0 else -1
        start = prev_end + 1
        values = t[start : end + 1]
        operational = float(group_time[start, end])
        segments.append(
            DPSegment(
                start=start,
                end=end,
                n=end - start + 1,
                bmi_min=float(b[start]),
```

```
                bmi_max=float(b[end]),
                raw_time=float(raw_time[start, end]),
                operational_time=operational,
                coverage=float(np.mean(values <= operational)),
                cost=float(cost[start, end]),
            )
        )
        end = prev_end
    segments.reverse()
    cuts = [
        float((segments[i].bmi_max + segments[i + 1].bmi_min) / 2)
        for i in range(len(segments) - 1)
    ]
    return DPPolicy(
        groups=groups,
        alpha=alpha,
        min_size=min_size,
        total_cost=float(dp[groups - 1, n - 1]),
        segments=segments,
        cutpoints=cuts,
        sorted_order=order,
    )



# ---------- 将动态规划切点取整为可执行规则并评估覆盖率 ----------
def rounded_policy_table(
    bmi: np.ndarray,
    tau: np.ndarray,
    fit: AFTFit,
    baseline: pd.DataFrame,
    *,
    cutpoints: Sequence[float],
    alpha: float,
    rounding: float = 0.5,
) -> tuple[pd.DataFrame, list[float]]:
    rounded = [round(float(c) / rounding) * rounding for c in cutpoints]
    rounded = sorted(set(rounded))
    bins = [-np.inf] + rounded + [np.inf]
    group_id = pd.cut(
        bmi, bins=bins, right=False, labels=False, include_lowest=True
    ).astype(int)
    rows: list[dict[str, Any]] = []
    for group in sorted(np.unique(group_id)):
```

```
        mask = group_id == group
        values = np.asarray(tau)[mask]
        time = max(10.0, exact_upper_quantile(values, alpha))
        subset = baseline.loc[mask].copy()
        probabilities = fit.cdf(time, subset)
        shortfall = np.maximum(values - time, 0)
        tail_n = max(1, int(np.ceil(0.10 * len(values))))
        cvar = float(np.sort(shortfall)[-tail_n:].mean())
        rows.append(
            {
                "group": int(group + 1),
                "bmi_left": float(np.min(np.asarray(bmi)[mask])),
                "bmi_right": float(np.max(np.asarray(bmi)[mask])),
                "n": int(mask.sum()),
                "recommended_week": time,
                "week_day": week_day(time),
                "coverage": float(np.mean(values <= time)),
                "mean_attainment_probability":
                float(np.mean(probabilities)),
                "uncovered_n": int(np.sum(values > time)),
                "out_of_window_n": int(np.sum(values > 25)),
                "cvar90_shortfall": cvar,
            }
        )
    return pd.DataFrame(rows), rounded


def determined_binary_at_time(
    intervals: pd.DataFrame, time: float
) -> tuple[np.ndarray, np.ndarray]:
    left = intervals["left"].to_numpy(float)
    right = intervals["right"].to_numpy(float)
    positive = right <= time
    negative = left >= time
    known = positive | negative
    labels = positive[known].astype(int)
    return known, labels


def support_status(values: np.ndarray) -> pd.DataFrame:
    array = np.asarray(values, dtype=float)
    return pd.DataFrame(
        {
            "minimum": [float(np.min(array))],
```

```
            "q01": [float(np.quantile(array, 0.01))],
            "q05": [float(np.quantile(array, 0.05))],
            "median": [float(np.median(array))],
            "q95": [float(np.quantile(array, 0.95))],
            "q99": [float(np.quantile(array, 0.99))],
            "maximum": [float(np.max(array))],
        }
    )
```

solve_baseline.py（主流程：问题一至问题四的基线求解串联）

```
# -*- coding: utf-8 -*-
#
================================================================================
# solve_baseline.py  问题一至问题四的基线求解主流程
# 功能：串联数据准备 -> 问题一分段线性混合模型 -> 问题二区间删失 AFT 与
#       动态规划分组 -> 问题三多因素 AFT 比较 -> 问题四女胎标签审计，
#       全部中间结果以 CSV/JSON 落盘，保证可复现。
# 说明：数据文件默认取相对路径“附件.xlsx”，不含本地绝对路径。
#
================================================================================

from __future__ import annotations

import argparse
import json
import os
import sys
import time
import warnings
from pathlib import Path

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
os.environ.setdefault("MKL_NUM_THREADS", "1")

import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy.special import expit
from scipy.stats import norm
from statsmodels.tools.sm_exceptions import ConvergenceWarning

```

```
from nipt_core import (
    SEED,
    PreparedData,
    classify_visit_states,
    construct_intervals,
    determined_binary_at_time,
    exact_dp_policy,
    fit_aft,
    prepare_data,
    rounded_policy_table,
    sha256_file,
    support_status,
    to_jsonable,
    week_day,
    write_json,
)



# ---------- 问题一：分段线性随机截距混合模型（logit 变换浓度） ----------
def fit_q1_model(
    visits: pd.DataFrame,
    *,
    knot: float | None = None,
    interaction: bool = False,
    age_height: bool = False,
    random_slope: bool = False,
):
    frame = visits.copy()
    frame["z_y"] = np.log(frame["y"] / (1 - frame["y"]))
    centers = {
        "ga": float(frame["ga"].mean()),
        "first_bmi": float(frame["first_bmi"].mean()),
        "age": float(frame["age"].mean()),
        "height": float(frame["height"].mean()),
    }
    frame["ga_c"] = frame["ga"] - centers["ga"]
    frame["first_bmi_c"] = frame["first_bmi"] - centers["first_bmi"]
    frame["age_c"] = frame["age"] - centers["age"]
    frame["height_c"] = frame["height"] - centers["height"]
    columns = ["ga_c", "first_bmi_c", "delta_bmi"]
    if knot is not None:
        name = f"ga_hinge_{int(knot)}"
        frame[name] = np.maximum(frame["ga"] - knot, 0)
```

```
        columns.append(name)
    if interaction:
        frame["ga_bmi_interaction"] = frame["ga_c"] *
        frame["first_bmi_c"]
        columns.append("ga_bmi_interaction")
    if age_height:
        columns.extend(["age_c", "height_c"])
    x = sm.add_constant(frame[columns], has_constant="add")
    exog_re = None
    if random_slope:
        exog_re = sm.add_constant(frame[["ga_c"]], has_constant="add")
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        result = sm.MixedLM(
            frame["z_y"],
            x,
            groups=frame[" 孕妇代码"],
            exog_re=exog_re,
        ).fit(
            reml=False,
            method="lbfgs",
            maxiter=1500,
            disp=False,
        )
    warning_text = [str(item.message) for item in caught]
    return result, frame, columns, centers, warning_text



# ---------- 问题一求解：候选结构比较、系数与预测网格输出 ----------
def q1_solve(data: PreparedData, out: Path) -> dict:
    visits = data.male_visits.copy()
    candidates: list[dict] = []
    fitted: dict[str, tuple] = {}
    specs = [("linear", None, False, False)]
    specs.extend((f"ga_hinge_{k}", float(k), False, False) for k in
    range(13, 19))
    specs.extend(
        [
            ("interaction", None, True, False),
            ("age_height", None, False, True),
        ]
    )
    for name, knot, interaction, age_height in specs:
```

```
        try:
            fit_tuple = fit_q1_model(
                visits,
                knot=knot,
                interaction=interaction,
                age_height=age_height,
            )
            result = fit_tuple[0]
            fitted[name] = fit_tuple
            candidates.append(
                {
                    "model": name,
                    "aic": result.aic,
                    "bic": result.bic,
                    "loglik": result.llf,
                    "converged": bool(result.converged),
                    "warnings": " | ".join(fit_tuple[-1]),
                }
            )
        except Exception as exc:
            candidates.append(
                {
                    "model": name,
                    "aic": np.nan,
                    "bic": np.nan,
                    "loglik": np.nan,
                    "converged": False,
                    "warnings": repr(exc),
                }
            )
    comparison = pd.DataFrame(candidates).sort_values("bic")
    comparison.to_csv(out / "q1_model_comparison.csv", index=False)
    chosen_name = str(comparison.dropna(subset=["bic"]).iloc[0]["model"])
    chosen, frame, columns, centers, warning_text = fitted[chosen_name]

    conf = chosen.conf_int()
    coefficient_rows = []
    for name in chosen.fe_params.index:
        coefficient_rows.append(
            {
                "term": name,
                "estimate_logit": chosen.fe_params[name],
                "std_error": chosen.bse_fe[name],
                "z": chosen.fe_params[name] / chosen.bse_fe[name],
```

```
            "p_value": chosen.pvalues[name],
            "ci_low": conf.loc[name, 0],
            "ci_high": conf.loc[name, 1],
            "odds_ratio_per_unit": np.exp(chosen.fe_params[name]),
        }
    )
coefficient_table = pd.DataFrame(coefficient_rows)
coefficient_table.to_csv(out / "q1_coefficients.csv", index=False)

random_intercept_var = float(chosen.cov_re.iloc[0, 0])
residual_var = float(chosen.scale)
icc = random_intercept_var / (random_intercept_var + residual_var)

grid_rows = []
for bmi in [28, 30, 32, 34, 36, 40]:
    for ga in [10, 11, 12, 14, 16, 18, 20, 22, 24, 25]:
        row = {
            "ga": ga,
            "first_bmi": bmi,
            "delta_bmi": 0.0,
            "age": centers["age"],
            "height": centers["height"],
        }
        values = {
            "ga_c": ga - centers["ga"],
            "first_bmi_c": bmi - centers["first_bmi"],
            "delta_bmi": 0.0,
            "age_c": 0.0,
            "height_c": 0.0,
            "ga_bmi_interaction": (ga - centers["ga"])
            * (bmi - centers["first_bmi"]),
        }
        for k in range(13, 19):
            values[f"ga_hinge_{k}"] = max(ga - k, 0)
        x = np.array([1.0] + [values[col] for col in columns])
        mean_logit = float(x @ chosen.fe_params.to_numpy())
        mean_y = float(expit(mean_logit))
        conditional_sd = np.sqrt(residual_var)
        marginal_sd = np.sqrt(residual_var + random_intercept_var)
        grid_rows.append(
            {
                **row,
                "predicted_y": mean_y,
                "predicted_logit": mean_logit,
```

```
                    "conditional_low": float(expit(mean_logit - 1.96 *
                    conditional_sd)),
                    "conditional_high": float(expit(mean_logit + 1.96 *
                    conditional_sd)),
                    "new_woman_low": float(expit(mean_logit - 1.96 *
                    marginal_sd)),
                    "new_woman_high": float(expit(mean_logit + 1.96 *
                    marginal_sd)),
                }
            )
    prediction_grid = pd.DataFrame(grid_rows)
    prediction_grid.to_csv(out / "q1_effect_grid.csv", index=False)

    random_slope = {}
    try:
        knot = float(chosen_name.rsplit("_", 1)[-1]) if "hinge" in
        chosen_name else None
        rs_fit, _, _, _, rs_warnings = fit_q1_model(
            visits,
            knot=knot,
            interaction=chosen_name == "interaction",
            age_height=chosen_name == "age_height",
            random_slope=True,
        )
        random_slope = {
            "aic": rs_fit.aic,
            "bic": rs_fit.bic,
            "loglik": rs_fit.llf,
            "converged": bool(rs_fit.converged),
            "cov_re": rs_fit.cov_re.to_numpy(),
            "residual_var": rs_fit.scale,
            "warnings": rs_warnings,
        }
    except Exception as exc:
        random_slope = {"error": repr(exc)}

    summary = {
        "chosen_model": chosen_name,
        "centers": centers,
        "n_visits": len(visits),
        "n_women": int(visits[" 孕妇代码"].nunique()),
        "aic": chosen.aic,
        "bic": chosen.bic,
        "loglik": chosen.llf,
```

```
        "converged": bool(chosen.converged),
        "warnings": warning_text,
        "random_intercept_var": random_intercept_var,
        "random_intercept_sd": np.sqrt(random_intercept_var),
        "residual_var_logit": residual_var,
        "residual_sd_logit": np.sqrt(residual_var),
        "icc_logit": icc,
        "random_slope_sensitivity": random_slope,
    }
    write_json(out / "q1_summary.json", summary)
    return summary


def interval_summary(intervals: pd.DataFrame) -> dict:
    return {
        "n_total": len(intervals),
        "counts": intervals["type"].value_counts(dropna=False).to_dict(),
        "state_conflicts": int(intervals["state_conflict"].sum()),
        "uncertain_visits": int(intervals["n_uncertain"].sum()),
        "uninformative_women": int((intervals["type"] ==
        "uninformative").sum()),
    }


def choose_policy_k(cost_table: pd.DataFrame) -> int:
    usable =
    cost_table.dropna(subset=["total_cost"]).sort_values("groups")
    cost1 = float(usable.loc[usable["groups"] == 1,
    "total_cost"].iloc[0])
    best = float(usable["total_cost"].min())
    denominator = max(cost1 - best, 1e-12)
    for _, row in usable.iterrows():
        if int(row["groups"]) < 2:
            continue
        captured = (cost1 - float(row["total_cost"])) / denominator
        if captured >= 0.90:
            return int(row["groups"])
    return int(usable.iloc[-1]["groups"])


# ---------- 问题二求解：多判定模式 -> AFT -> 动态规划分组排程 ----------
def q2_solve(data: PreparedData, out: Path) -> dict:
    modes: dict[str, pd.DataFrame] = {}
```

```
    hard_states = classify_visit_states(
        data.male_visits, data.sigma_tech, mode="hard"
    )
    modes["hard"] = construct_intervals(hard_states, data.male_baseline)
    credible_states = classify_visit_states(
        data.male_visits,
        data.sigma_tech,
        mode="credible",
        eta=0.025,
    )
    modes["credible_eta025"] = construct_intervals(
        credible_states, data.male_baseline
    )
    for eta in [0.01, 0.05, 0.10]:
        states = classify_visit_states(
            data.male_visits,
            data.sigma_tech,
            mode="credible",
            eta=eta,
        )
        modes[f"credible_eta{str(eta).replace('.', '')}"] =
        construct_intervals(
            states, data.male_baseline
        )
    for name, intervals in modes.items():
        intervals.to_csv(out / f"q2_intervals_{name}.csv", index=False)

    fit_rows: list[dict] = []
    fit_map = {}
    for mode_name in ["hard", "credible_eta025"]:
        intervals = modes[mode_name]
        for distribution in ["lognormal", "loglogistic", "weibull"]:
            fit = fit_aft(
                intervals,
                ["first_bmi"],
                distribution=distribution,
                nonnegative_features=["first_bmi"],
            )
            fit_map[(mode_name, distribution)] = fit
            fit_rows.append({"mode": mode_name, **fit.to_dict()})
    aft_comparison = pd.DataFrame(fit_rows)
    aft_comparison.to_csv(out / "q2_aft_comparison.csv", index=False)

    q_grid_rows = []
```

```
    for (mode_name, distribution), fit in fit_map.items():
        grid = pd.DataFrame({"first_bmi": [28.5, 30.5, 32, 34, 36, 40,
        42, 46.875]})
        for rho in [0.80, 0.90, 0.95]:
            values = fit.quantile(rho, grid)
            for bmi, value in zip(grid["first_bmi"], values):
                q_grid_rows.append(
                    {
                        "mode": mode_name,
                        "distribution": distribution,
                        "rho": rho,
                        "bmi": bmi,
                        "quantile_week": value,
                        "week_day": week_day(value),
                        "within_10_25": 10 <= value <= 25,
                    }
                )
    pd.DataFrame(q_grid_rows).to_csv(out / "q2_aft_quantile_grid.csv",
    index=False)

    primary_fit = min(
        [
            fit_map[("hard", distribution)]
            for distribution in ["lognormal", "loglogistic", "weibull"]
        ],
        key=lambda item: item.aic,
    )
    credible_fit = min(
        [
            fit_map[("credible_eta025", distribution)]
            for distribution in ["lognormal", "loglogistic", "weibull"]
        ],
        key=lambda item: item.aic,
    )

    baseline = data.male_baseline.sort_values("孕妇代
    码").reset_index(drop=True)
    bmi = baseline["first_bmi"].to_numpy(float)
    policies_summary = []
    chosen_tables = {}
    for model_name, fit in [("hard", primary_fit), ("credible",
    credible_fit)]:
        for rho in [0.80, 0.90]:
            tau = fit.quantile(rho, baseline)
```

```
            cost_rows = []
            policy_map = {}
            for groups in range(1, 7):
                try:
                    policy = exact_dp_policy(
                        bmi,
                        tau,
                        groups=groups,
                        alpha=rho,
                        min_size=20,
                    )
                    policy_map[groups] = policy
                    cost_rows.append(
                        {
                            "model": model_name,
                            "rho": rho,
                            "groups": groups,
                            "total_cost": policy.total_cost,
                            "cutpoints": json.dumps(policy.cutpoints),
                        }
                    )
                except Exception:
                    cost_rows.append(
                        {
                            "model": model_name,
                            "rho": rho,
                            "groups": groups,
                            "total_cost": np.nan,
                            "cutpoints": "[]",
                        }
                    )
            costs = pd.DataFrame(cost_rows)
            selected_k = choose_policy_k(costs)
            selected = policy_map[selected_k]
            table, rounded = rounded_policy_table(
                bmi,
                tau,
                fit,
                baseline,
                cutpoints=selected.cutpoints,
                alpha=rho,
            )
            table.insert(0, "model", model_name)
            table.insert(1, "rho", rho)
```

```
            table.insert(2, "selected_k", selected_k)
            table["raw_cutpoints"] = json.dumps(selected.cutpoints)
            table["rounded_cutpoints"] = json.dumps(rounded)
            key = f"{model_name}_rho{int(rho * 100)}"
            table.to_csv(out / f"q2_policy_{key}.csv", index=False)
            costs.to_csv(out / f"q2_policy_costs_{key}.csv", index=False)
            chosen_tables[key] = table
            policies_summary.append(
                {
                    "key": key,
                    "selected_k": selected_k,
                    "raw_cutpoints": selected.cutpoints,
                    "rounded_cutpoints": rounded,
                    "total_cost": selected.total_cost,
                    "out_of_window_individuals": int(np.sum(tau > 25)),
                }
            )

    state_sensitivity = {
        name: interval_summary(intervals) for name, intervals in
        modes.items()
    }
    summary = {
        "intervals": state_sensitivity,
        "primary_point_model": primary_fit.to_dict(),
        "credible_point_model": credible_fit.to_dict(),
        "policies": policies_summary,
    }
    write_json(out / "q2_summary.json", summary)
    return summary



# ---------- 问题三求解：多因素 AFT 候选集与样本内校准 ----------
def q3_solve(data: PreparedData, out: Path) -> dict:
    hard_states = classify_visit_states(
        data.male_visits, data.sigma_tech, mode="hard"
    )
    intervals = construct_intervals(hard_states, data.male_baseline)
    candidate_features = {
        "bmi_only": ["first_bmi"],
        "bmi_age_height": ["first_bmi", "age", "height"],
        "bmi_full": [
            "first_bmi",
```

```
            "age",
            "height",
            "gravidity",
            "parity",
            "ivf_iui",
        ],
        "weight_height_full": [
            "first_weight",
            "height",
            "age",
            "gravidity",
            "parity",
            "ivf_iui",
        ],
    }
    fits = {}
    rows = []
    calibration_rows = []
    for name, features in candidate_features.items():
        nonnegative = ["first_bmi"] if "first_bmi" in features else []
        fit = fit_aft(
            intervals,
            features,
            distribution="lognormal",
            nonnegative_features=nonnegative,
            ridge=0.0,
        )
        fits[name] = fit
        rows.append({"model": name, **fit.to_dict()})
        for time_point in [12, 14, 16, 18, 20]:
            known, labels = determined_binary_at_time(intervals,
            time_point)
            subset = intervals.loc[known].copy()
            probabilities = fit.cdf(time_point, subset)
            calibration_rows.append(
                {
                    "model": name,
                    "time": time_point,
                    "n_known": int(known.sum()),
                    "prevalence": float(labels.mean()),
                    "mean_predicted": float(probabilities.mean()),
                    "brier_in_sample": float(np.mean(np.square(labels -
                    probabilities))),
                }
```

```
            )
    comparison = pd.DataFrame(rows).sort_values("bic")
    comparison.to_csv(out / "q3_aft_comparison.csv", index=False)
    pd.DataFrame(calibration_rows).to_csv(
        out / "q3_time_calibration_in_sample.csv", index=False
    )
    best_name = str(comparison.iloc[0]["model"])
    summary = {
        "best_bic_model": best_name,
        "bmi_only": fits["bmi_only"].to_dict(),
        "best": fits[best_name].to_dict(),
        "bic_improvement_vs_bmi": fits["bmi_only"].bic -
        fits[best_name].bic,
        "aic_improvement_vs_bmi": fits["bmi_only"].aic -
        fits[best_name].aic,
    }
    write_json(out / "q3_summary.json", summary)
    return summary



# ---------- 问题四前置：女胎阳性标签与 Z 值阈值审计 ----------
def female_audit(data: PreparedData, out: Path) -> dict:
    female = data.female_raw.copy()
    label_text = female[" 染色体的非整倍体"].fillna("").astype(str)
    audit = {
        "records": len(female),
        "women": int(female[" 孕妇代码"].nunique()),
        "abnormal_records": int(label_text.ne("").sum()),
        "abnormal_women": int(
            female.loc[label_text.ne(""), " 孕妇代码"].nunique()
        ),
        "gc_outside_40_60": int(
            ((female["GC 含量"] < 0.40) | (female["GC 含量"] >
            0.60)).sum()
        ),
        "labels": {},
    }
    rows = []
    for chrom in [13, 18, 21]:
        y = label_text.str.contains(f"T{chrom}", regex=False)
        z = female[f"{chrom} 号染色体的 Z 值"].to_numpy(float)
        audit["labels"][f"T{chrom}"] = {
            "positive_records": int(y.sum()),
```

```
            "positive_women": int(female.loc[y, " 孕妇代码"].nunique()),
            "z_ge_3_sensitivity": float(np.mean(z[y] >= 3)),
            "abs_z_ge_3_sensitivity": float(np.mean(np.abs(z[y]) >= 3)),
            "positive_z_min": float(np.min(z[y])),
            "positive_z_max": float(np.max(z[y])),
        }
        rows.append({"label": f"T{chrom}",
        **audit["labels"][f"T{chrom}"]})
    pd.DataFrame(rows).to_csv(out / "q4_label_audit.csv", index=False)
    write_json(out / "q4_audit_summary.json", audit)
    return audit


# ---------- 主入口：解析参数、串联四问、落盘全部结果 ----------
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--data", type=Path, default=Path(" 附件.xlsx"))
    parser.add_argument(
        "--output", type=Path,
        default=Path("nipt_solution/outputs/baseline")
    )
    args = parser.parse_args()
    args.output.mkdir(parents=True, exist_ok=True)
    started = time.time()
    data = prepare_data(args.data)
    write_json(args.output / "data_audit.json", data.technical_summary)
    support_status(data.male_baseline["first_bmi"].to_numpy()).to_csv(
        args.output / "male_bmi_support.csv", index=False
    )
    summaries = {
        "metadata": {
            "seed": SEED,
            "data": str(args.data),
            "data_sha256": sha256_file(args.data),
            "python": sys.version,
        },
        "q1": q1_solve(data, args.output),
        "q2": q2_solve(data, args.output),
        "q3": q3_solve(data, args.output),
        "q4_audit": female_audit(data, args.output),
    }
    summaries["metadata"]["elapsed_seconds"] = time.time() - started
    write_json(args.output / "baseline_manifest.json", summaries)
```

```
    print(json.dumps(to_jsonable(summaries), ensure_ascii=False,
    indent=2))


if __name__ == "__main__":
    main()
```